

RAMA:	INFORMÁTICA	CICLO:	DESENVOLVEMENTO DE APLICACIÓNS MULTIPLATAFORMA					
MÓDULO:	ACCESO A DATOS					CURSO:		2º
PROFESOR:	IAGO DOVAL IGLESIAS							
AVALIACIÓN:	SEGUNDA	GRUPO:	ÚNICO	DATA:	16/03/2026			

NOMBRE ALUMNO:	Xela	DNI:	3.4
----------------	------	------	-----

El examen consta de dos partes valoradas en 5 puntos cada una. Para aprobar el examen hay que **sacar un 5** entre las dos partes teniendo que obtener, **como mínimo, un 2** en cada una de ellas.

Una vez terminado el examen se ha de subir **un único fichero comprimido** al aula virtual con todos los archivos java que se hayan utilizado. El nombre del fichero tendrá el formato: **1ºApellido_2ºApellido_Nombre**.

Ejercicios Conectores (5 pt) 2.7

Para la realización de los ejercicios de conectores se usará el fichero **Conectores.java** junto (los métodos que ya están creados en este fichero se pueden modificar). Se adjunta además el conector necesario para la realización de la conexión y el fichero **admarzo26.txt** con las sentencias para crear la base de datos admarzo26 y las tablas que se usaran en los ejercicios.

Se **han de tratar**, de forma adecuada, **las excepciones** que puedan surgir y **liberar todos los recursos** utilizados (-0.10pt por cada elemento no tratado).

Crea los siguientes métodos para los que se ha **de mostrar un ejemplo de su invocación**. (-0.15pt por cada ejercicio en el que no se indique).

- 1) (1.4 pt) Método, usando sentencias **preparadas**, que nos permita mostrar, ordenado por posición, la posición alcanzada en la clasificación, el gap con el piloto precedente, el nombre de la escudería y el nombre del piloto de los pilotos que estén entre dos posición pasadas como parámetros. Debe devolver el número de elementos encontrados o -1 si se ha producido un error y visualizarlo fuera del método. *No es int, tu devuelves 0*
Si no puedes obtener el nombre de la escudería puedes mostrar su código pero se valorará -0.4 pt menos.
- 2) (1.2 pt) ¡SE HA SANCIONADO A UN PILOTO POR UNA DECLARACIONES POLÉMICAS! El susodicho se cree superior a todos sus rivales por lo que la FIA le sanciona a la última posición de la parrilla.
Crea un método que aumente en 7 posiciones la clasificación, a los pilotos que hayan dado 12 vueltas y cuyo nombre contenga al menos dos "o". El método deberá devolver un valor indicado si la actualización se ha llevado a cabo(1), si no (0) o si se ha producido un error (-1).
- 3) (1 pt) Crea un método, usando sentencias sin preparar, que nos devuelva el nombre de piloto peor clasificado, junto con la frase "no va a ganar NUNCA la 33!". Imprime este mensaje en el main. *con return*
- 4) (1.4 pt) Método al que se le pasa el nombre de una tabla y muestre de esa tabla y para cada columna: el nombre de la columna (COLUMN_NAME), su tipo de datos (TYPE_NAME), si permite nulos (NULLABLE) y si es autoincrementado (IS_AUTOINCREMENT). Para estas dos últimas, deberá aparecer "Sí" o "No".

RAMA:	INFORMÁTICA	CICLO:	DESENVOLVEMENTO DE APLICACIÓNS MULTIPLATAFORMA				
MÓDULO:	ACCESO A DATOS				CURSO:		2º
PROFESOR:	IAGO DOVAL IGLESIAS						
AVALIACIÓN:	SEGUNDA	GRUPO:	ÚNICO	DATA:	16/03/2026		

Ejercicios Servicios Web (5 pt)

Usando la base de datos creada mediante el fichero admarzo26.txt realiza los métodos siguientes teniendo en cuenta los siguientes aspectos:

- El path inicial del servicio web será **api**. *— 0.2*
- Si en los ejercicios no se especifica un path se usará el path del punto anterior.
- Todos los métodos han de devolver un objeto tipo Response. En el caso de producirse algún error o excepción se ha de devolver un response con un código de error y un mensaje explicativo.
- Si no se especifica lo contrario se produce y/o consume JSON y XML. En el caso de listas solo se produce JSON.
- Se tiene que crear un ArrayList estático que contendrá las escuderías para usarse en los ejercicios 5 y 6. Se tiene que llamar escuderias.

5) (1 pt) Un método que cree y añada al ArrayList escuderias una nueva escudería. Los parámetros sede y su año de debut se cubren con QueryParams mientras que el resto se cubren "a mano" (No se introduce nada en el Body). El valor por defecto de sede debe ser "Desconocida". Deberá devolver esta escudería. No hay que controlar que existan escuderías con códigos repetidos. *No consume / Produce / No devuelve nada*

6) (1.10 pt) Un método que en el path "{motor}" borre, del ArrayList escuderias, todas las escuderías con un motorista que contenga el valor pasado. Debe devolver la lista de escuderías. *Mal QueryParam*

7) (1.30 pt) Un método que devuelva el nombre de todas las escuderías presentes en la base de datos. *Altes mal consumo, No devuelve nada, Mal mensaje error*

8) (1.60 pt) Crea un método en el path "{actualizar}", que al pasarle una escudería mediante un JSON, la actualice si ya existe su codEscuderia en la base de datos. Si no existe, que la inserte.

Si no puedes insertar la escudería cuando no existe el CodEscuderia, puede saltar un mensaje de Error pero se valorará -0.8 pt menos.